Introduzione ai Sistemi Operativi

Fondamenti e Architettura

Prof. Fedeli Massimo - Corso ITS – Fabbrica Digitale

Cos'è un Sistema Operativo

Un Sistema Operativo (SO) è:

- Un insieme di **programmi** che gestisce le **risorse di un computer**
- Un intermediario tra **utente** e **hardware**
- Un ambiente di sviluppo ed esecuzione per le **applicazioni**

Cosa sono le risorse del computer?
Le risorse sono tutti i componenti hardware e software necessari all'esecuzione dei programmi.

- **CPU**: esegue le istruzioni dei programmi
- **Memoria RAM**: contiene dati e programmi in esecuzione
- **Memoria secondaria**: dischi SSD/HDD dove sono memorizzati i file
- **Dispositivi di I/O**: tastiera, mouse, stampante, rete, ecc.

Cos'è un Sistema Operativo

Il sistema operativo decide **come e quando assegnare queste risorse** ai programmi in esecuzione.

Esempi di sistemi operativi

- Linux
- Windows
- macOS
- Android

Ruolo del Sistema Operativo

Il sistema operativo:

- controlla la CPU
- gestisce la memoria
- gestisce i dispositivi di input/output
- gestisce file e dati

Struttura di un Sistema di Calcolo

Utenti
↓
Applicazioni
↓
Sistema Operativo
↓
Hardware

Il concetto di astrazione

Il sistema operativo fornisce un **livello di astrazione** sopra l'hardware, cioè nasconde la complessità dei componenti fisici del computer e presenta agli utenti e ai programmi un'interfaccia più semplice da usare.

Struttura di un Sistema di Calcolo

Esempi di astrazione

- **File**: i dati su disco vengono gestiti come file e cartelle, senza preoccuparsi dei settori fisici del disco.
- **Processi**: un programma in esecuzione viene visto come un processo, senza gestire direttamente CPU e registri.
- **Memoria virtuale**: ogni programma ha l'illusione di avere a disposizione tutta la memoria del sistema.
- **Dispositivi**: tastiere, dischi o stampanti sono accessibili tramite comandi standard, indipendentemente dall'hardware reale.

Grazie all'astrazione, i programmatori possono sviluppare software senza conoscere nei dettagli l'hardware della macchina.

Risorse gestite dal Sistema Operativo

Principali risorse:

- CPU
- Memoria RAM
- Memoria secondaria (dischi)
- Dispositivi di I/O
- Rete

Il SO decide come assegnare queste risorse ai programmi.

Obiettivi principali di un Sistema Operativo

Un sistema operativo deve garantire diversi obiettivi fondamentali per permettere al sistema di funzionare in modo efficiente e sicuro.

- **Efficienza nell'uso delle risorse**
  Il sistema operativo deve utilizzare al meglio le risorse disponibili (CPU, memoria, disco e dispositivi di I/O), evitando sprechi e riducendo i tempi di inattività del sistema.
- **Sicurezza**
  Il sistema operativo deve proteggere il sistema e i dati da accessi non autorizzati, garantendo meccanismi di autenticazione, permessi di accesso e isolamento tra processi e utenti.
- **Affidabilità**
  Il sistema deve continuare a funzionare correttamente anche in presenza di errori hardware o software, evitando crash e perdita di dati.
- **Condivisione delle risorse**
  In sistemi multiutente o multiprogrammati più programmi devono poter utilizzare le stesse risorse (CPU, memoria, periferiche) senza

Esempi pratici degli obiettivi di un Sistema Operativo

- **Efficienza**
  In un computer moderno è possibile eseguire contemporaneamente browser, editor di testo e musica. Il sistema operativo assegna dinamicamente la CPU e la memoria a ciascun programma.
- **Sicurezza**
  In Linux o Windows ogni utente ha un proprio account e permessi di accesso ai file. Un utente normale non può modificare file di sistema senza privilegi di amministratore.
- **Affidabilità**
  Se un'applicazione si blocca (ad esempio un browser), il sistema operativo può terminare il processo senza bloccare tutto il sistema.
- **Condivisione delle risorse**
  Più programmi possono utilizzare contemporaneamente la stessa stampante o la stessa connessione di rete.
- **Facilità d'uso**
  Le interfacce grafiche (Windows, macOS, GNOME, KDE) permettono di gestire file e programmi tramite finestre, icone e menu.

Evoluzione dei Sistemi Operativi

Le principali fasi storiche:

1. Prima generazione (anni 50)
2. Sistemi batch
3. Multiprogrammazione
4. Time sharing
5. Sistemi moderni

Prima generazione (anni 50)

Caratteristiche:

- Programmazione in linguaggio macchina
- Input tramite schede perforate
- Nessun sistema operativo

Ogni programma controllava direttamente l'hardware.

Sistemi Batch

Batch significa letteralmente *lotto* o *gruppo*.
Nei primi sistemi informatici i programmi venivano raccolti in gruppi (batch) ed eseguiti automaticamente uno dopo l'altro.

Come funzionava

- Gli utenti preparavano i propri programmi (detti **job**).
- I programmi venivano consegnati all'operatore del computer, spesso tramite **schede perforate** o nastri magnetici.
- L'operatore caricava un **lotto di job** nel sistema.
- Il sistema operativo eseguiva i job **in sequenza**, senza alcuna interazione con l'utente.

Caratteristiche principali

- Nessuna interazione diretta con il computer
- Esecuzione automatica dei programmi
- Ogni job iniziava solo dopo la terminazione del precedente

Esempio di Job Batch

```
JOB BEGIN
$compile
$load
$run
JOB END
```

Il sistema operativo eseguiva automaticamente i job.

Limiti dei sistemi Batch

Problemi principali:

- CPU spesso inattiva durante I/O
- nessuna interazione con l'utente
- esecuzione sequenziale dei job

Spooling

SPooling =
Simultaneous Peripheral Operation On Line
Idea principale:

- usare il disco come buffer
- sovrapporre I/O ed elaborazione

Multiprogrammazione

Nei sistemi batch semplici la CPU rimaneva spesso inattiva quando un programma era in attesa di operazioni di I/O (lettura da disco, stampa, accesso a dispositivi).

Idea della multiprogrammazione

- Più programmi (**job**) vengono caricati contemporaneamente nella memoria principale.
- Il sistema operativo mantiene un insieme di programmi pronti per l'esecuzione.
- Quando un programma deve attendere un'operazione di I/O, la CPU viene assegnata ad un altro programma.

Obiettivo

- Ridurre i tempi di inattività della CPU
- Aumentare l'utilizzo delle risorse del sistema
- Migliorare le prestazioni complessive del computer

Multiprogrammazione: utilizzo della CPU

Nei sistemi multiprogrammati più programmi condividono la CPU. Quando un programma è in attesa di un'operazione di I/O, il sistema operativo assegna la CPU ad un altro programma.

Esempio di utilizzo della CPU nel tempo

```
tempo →
Job 1 : [CPU][CPU][ I/O ][ I/O ][ I/O ][CPU][CPU] Job 2 : [CPU
I/O ][ I/O ] Job 3 : [CPU][CPU][CPU][CPU]
```

Risultato

- la CPU rimane attiva per la maggior parte del tempo
- più programmi avanzano nella loro esecuzione
- il sistema utilizza meglio le risorse disponibili

Scheduling

Lo scheduling decide:

- quale processo usare la CPU
- quando cambiare processo

Tipi principali:

- Long term scheduling
- Short term scheduling

Time Sharing

Il **time sharing** è un'evoluzione dei sistemi multiprogrammati che introduce l'**interazione diretta tra utente e computer**.
Nei primi sistemi informatici l'utente non interagiva direttamente con il computer: i programmi venivano eseguiti in modalità batch.
Con il time sharing diventa possibile lavorare **interattivamente** con il sistema.

Caratteristiche principali

- più utenti possono utilizzare il sistema contemporaneamente
- ogni utente può eseguire uno o più programmi
- il sistema operativo gestisce la condivisione delle risorse

Divisione della CPU nel Time Sharing

Per permettere a più utenti di utilizzare il sistema, la CPU viene suddivisa in piccoli intervalli di tempo.
Questi intervalli sono chiamati:

time slice oppure **quanto di tempo**

Il sistema operativo assegna la CPU ad un processo per un breve intervallo di tempo.
Quando il tempo termina:

- il sistema operativo interrompe il processo corrente
- salva il suo stato
- assegna la CPU ad un altro processo

Questo meccanismo è gestito dallo **scheduling della CPU**.

Effetto del Time Sharing

L'alternanza molto rapida dei processi produce un effetto importante.
Ogni utente ha l'impressione di avere un computer dedicato.
In realtà:

- la CPU viene condivisa tra molti processi
- il sistema operativo alterna rapidamente l'esecuzione
- ogni processo riceve periodicamente tempo di CPU

Questo permette di ottenere:

- sistemi multiutente
- buona interattività
- utilizzo efficiente delle risorse

Esempi di sistemi time-sharing moderni:

- Unix
- Linux
- macOS

Time Slice

Time slice = intervallo di tempo assegnato a un processo.
Quando scade:

- avviene un context switch
- la CPU passa ad un altro processo

Context Switch

Il **context switch** (cambio di contesto) è l'operazione con cui il sistema operativo sospende l'esecuzione di un processo e assegna la CPU ad un altro processo.

Operazioni principali

- **Salvataggio dello stato del processo corrente** (registri CPU, contatore di programma, informazioni di memoria).
- Memorizzazione dello stato nella struttura dati del sistema operativo chiamata **Process Control Block (PCB)**.
- **Caricamento dello stato del processo successivo** che deve utilizzare la CPU.

Il processo selezionato riprende l'esecuzione esattamente dal punto in cui era stato interrotto.

Quando avviene un Context Switch

Il cambio di contesto può avvenire in diverse situazioni:

- scadenza del **quanto di tempo** nei sistemi time-sharing
- richiesta di **operazioni di I/O** da parte del processo
- arrivo di un **interrupt**
- terminazione di un processo

Overhead

Il context switch introduce un costo computazionale perché la CPU deve:

- salvare e ripristinare registri e stato del processo
- aggiornare le strutture dati del sistema operativo
- eseguire codice del kernel

Durante queste operazioni la CPU non esegue lavoro utile per i programmi utente.

Architettura di un sistema di calcolo

Un sistema di calcolo è composto da diversi componenti hardware che collaborano per eseguire i programmi.
Componenti principali:

- **CPU (Central Processing Unit)**: esegue le istruzioni dei programmi.
- **Memoria principale (RAM)**: contiene dati e programmi in esecuzione.
- **Controller dei dispositivi**: componenti hardware che gestiscono le periferiche.
- **Bus di sistema**: canale di comunicazione tra CPU, memoria e dispositivi.

Il sistema operativo coordina l'utilizzo di queste componenti per garantire il corretto funzionamento del sistema.

Controller dei dispositivi

Le periferiche non comunicano direttamente con la CPU.
Ogni dispositivo è gestito da un **controller**, cioè un componente hardware specializzato che controlla il dispositivo.
Funzioni principali:

- ricevere comandi dalla CPU
- controllare il funzionamento del dispositivo
- trasferire dati tra dispositivo e memoria

Esempi:

- controller del disco
- controller video
- controller di rete

Interrupt

Un **interrupt** è un segnale che interrompe temporaneamente l'esecuzione della CPU per segnalare il verificarsi di un evento.
Gli interrupt permettono al sistema operativo di reagire rapidamente a eventi provenienti da hardware o software.
Gli interrupt possono essere generati da:

- dispositivi hardware
- programmi in esecuzione

Interrupt Hardware

Gli **interrupt hardware** sono generati dai dispositivi per segnalare eventi alla CPU.
Esempi:

- completamento di una operazione di lettura da disco
- pressione di un tasto sulla tastiera
- arrivo di dati dalla rete

Quando avviene l'evento, il dispositivo invia un segnale alla CPU che interrompe temporaneamente il programma in esecuzione.

Interrupt Software

Gli **interrupt software** sono generati dai programmi in esecuzione.
Possono essere causati da:

- errori durante l'esecuzione di un programma
- accessi illegali alla memoria
- richieste di servizi al sistema operativo

Un esempio importante di interrupt software è la **system call**, con cui un programma richiede un servizio al sistema operativo.

Gestione degli interrupt

Quando arriva un interrupt il sistema operativo esegue una sequenza di operazioni.

1. la CPU salva lo stato del processo corrente
2. il controllo passa alla **routine di servizio dell'interrupt**
3. viene gestito l'evento che ha generato l'interrupt
4. lo stato del processo viene ripristinato
5. il processo riprende l'esecuzione

Questo meccanismo permette al sistema di reagire rapidamente agli eventi.

Input / Output

Le operazioni di **Input/Output (I/O)** permettono la comunicazione tra il computer e i dispositivi esterni.
Esempi di dispositivi di I/O:

- tastiera
- disco
- stampante
- rete

Le operazioni di I/O sono generalmente più lente rispetto alle operazioni della CPU.

I/O sincrono

Nel caso di **I/O sincrono** il processo che richiede l'operazione rimane bloccato fino al completamento

- il processo non può proseguire
- la CPU può assegnare il tempo ad altri processi

Esempio:
lettura di un file dal disco.

I/O asincrono

Nel caso di **I/O asincrono** il processo continua l'esecuzione mentre l'operazione di I/O viene svolta.
Il completamento dell'operazione viene segnalato tramite un interrupt.
Vantaggi:

- migliore utilizzo della CPU
- possibilità di eseguire più operazioni in parallelo

DMA

DMA (Direct Memory Access) è una tecnica che permette ai dispositivi di trasferire dati direttamente tra dispositivo e memoria.
Senza DMA:

- la CPU deve gestire ogni trasferimento di dati

Con DMA:

- il trasferimento avviene senza coinvolgere la CPU
- la CPU può eseguire altre operazioni

Vantaggi:

- minore carico sulla CPU
- maggiore velocità nei trasferimenti

Protezione del sistema

Nei sistemi multiprogrammati o multiutente è necessario proteggere le risorse del sistema.
Le principali risorse da proteggere sono:

- memoria
- processi
- file
- dispositivi

Il sistema operativo impedisce che un programma interferisca con il funzionamento degli altri.

Dual Mode

Molte architetture hardware prevedono due modalità di esecuzione.

- **User Mode**: esecuzione dei programmi utente
- **Kernel Mode**: esecuzione del sistema operativo

Solo il kernel può eseguire **istruzioni privilegiate** come l'accesso ai dispositivi o la gestione della memoria.
Questo meccanismo aumenta la sicurezza del sistema.

System Call

Le **system call** sono l'interfaccia tra programmi utente e sistema operativo.
Permettono ai programmi di richiedere servizi al kernel.
Esempi di system call:

- open()
- read()
- write()
- fork()

Esempio Linux

Esempio di system call:

```
read(fd, buffer, size);
```

Il programma richiede al kernel di leggere dati associati al file descriptor `fd` e salvarli nel buffer di memoria.

Processo

Un **processo** è un programma in esecuzione.
Oltre al codice del programma include:

- dati del programma
- registri della CPU
- contatore di programma
- informazioni sullo stato di esecuzione

Il processo rappresenta l'unità fondamentale di esecuzione nel sistema operativo.

Gestione dei processi

Il sistema operativo gestisce l'intero ciclo di vita dei processi.
Compiti principali:

- creazione dei processi
- terminazione dei processi
- sospensione e ripresa dei processi
- sincronizzazione tra processi

Queste operazioni sono gestite tramite strutture dati interne al sistema operativo.

Gestione della memoria

La memoria principale è una risorsa limitata che deve essere condivisa tra più processi.
Il sistema operativo deve:

- allocare memoria ai processi
- liberare memoria quando non serve più
- proteggere gli spazi di memoria

Memoria Virtuale

La **memoria virtuale** permette di utilizzare uno spazio di memoria logico più grande della memoria fisica disponibile.
Il sistema operativo utilizza il disco come estensione della memoria.
Vantaggi:

- esecuzione di programmi di grandi dimensioni
- migliore gestione della memoria

Gestione dei File

Il sistema operativo fornisce una visione astratta della memoria secondaria tramite il concetto di **file**.
Funzioni principali:

- creazione e cancellazione dei file
- organizzazione in directory
- gestione dei permessi di accesso

File System

Il **file system** definisce come i dati sono organizzati e memorizzati sui dispositivi.
Esempi di file system:

- NTFS
- ext4
- FAT32
- APFS

Ogni sistema operativo supporta uno o più file system.

Gestione I/O

Il sistema operativo gestisce le operazioni di input/output tramite diversi meccanismi.

- driver di dispositivo
- buffer per memorizzazione temporanea
- code di richieste

Questi strumenti permettono di gestire più richieste contemporaneamente.

Driver di dispositivo

Un **driver** è un software che permette al sistema operativo di comunicare con un dispositivo hardware.
Esempi:

- driver stampante
- driver scheda video
- driver scheda di rete

I driver nascondono i dettagli specifici dell'hardware.

Sicurezza

La sicurezza del sistema operativo serve a proteggere il sistema e i dati.
Minacce principali:

- accessi non autorizzati
- attacchi di rete
- malware

Il sistema operativo utilizza meccanismi di autenticazione e controllo degli accessi.

Interfaccia utente

L'interfaccia utente permette agli utenti di interagire con il sistema operativo.
Due tipi principali:

- CLI (Command Line Interface)
- GUI (Graphical User Interface)

CLI

La CLI permette di interagire con il sistema tramite comandi testuali.
Esempio Linux:

```
ls
cd
mkdir
```

GUI

La GUI permette di interagire con il sistema tramite elementi grafici.

- finestre
- icone
- menu
- mouse

Componenti di un Sistema Operativo

Principali moduli del sistema operativo:

- gestione dei processi
- gestione della memoria
- gestione dei file
- gestione dell'I/O
- sicurezza

Struttura Monolitica

In una struttura monolitica il sistema operativo è un unico grande programma.
Esempi storici:

- MS-DOS
- prime versioni di UNIX

Struttura a livelli

In una struttura a livelli il sistema operativo è organizzato in livelli gerarchici.
Ogni livello utilizza i servizi del livello sottostante.

Esempio di struttura a livelli

Livelli tipici:

- hardware
- gestione CPU
- gestione memoria
- gestione I/O
- programmi utente

Kernel

Il **kernel** è il nucleo del sistema operativo.
Gestisce le operazioni fondamentali:

- gestione dei processi
- gestione della memoria
- gestione degli interrupt
- gestione delle system call

Microkernel

Nel modello microkernel solo le funzioni essenziali sono presenti nel kernel. Altre funzionalità vengono eseguite come processi separati.

Vantaggi del Microkernel

- maggiore modularità
- maggiore sicurezza
- maggiore affidabilità

Svantaggi del Microkernel

- maggiore comunicazione tra processi
- possibile riduzione delle prestazioni

Esempi di Sistemi Operativi

- Linux
- Windows
- macOS
- Android
- iOS

Conclusioni

Il sistema operativo è il componente fondamentale di ogni computer.
Permette di:

- gestire le risorse hardware
- eseguire applicazioni
- garantire sicurezza e stabilità

Riferimenti Bibliografici

- A. Silberschatz, P. B. Galvin, G. Gagne
  Operating System Concepts
  Wiley, ultima edizione.
- A. S. Tanenbaum, H. Bos
  Modern Operating Systems
  Pearson.
- W. Stallings
  Operating Systems: Internals and Design Principles
  Pearson.
- R. Love
  Linux Kernel Development
  Addison-Wesley.
- A. Ciampolini
  Sistemi Operativi – Materiale didattico
  Università di Bologna.